

TAREA 1

GYMTRACK

Luis Moyano San Bruno

1. URL del repositorio de control de versiones usado

Para el desarrollo de GymTrack se utilizará un repositorio de GitHub.

URL del repositorio:

<https://github.com/lmoyansa/gymtrack>

En el repositorio se almacenarán el código fuente de la aplicación y los principales documentos generados durante el proyecto. Git permitirá conservar el historial de cambios mediante commits.

Notion se utilizará para organizar las historias de usuario, las tareas de cada sprint y su estado.

2. Justificación de la plataforma usada

Se ha elegido GitHub porque permite utilizar Git, trabajar con diferentes ramas y consultar fácilmente la evolución del proyecto.

Aunque el proyecto será individual, se seguirá una organización de ramas similar a la utilizada en proyectos con varios desarrolladores.

GitHub permitirá:

- Conservar el historial de cambios del proyecto.
- Separar el desarrollo activo de las versiones estables.
- Mantener accesibles el código y la documentación.
- Facilitar la revisión del proyecto por parte del profesor.

El seguimiento del trabajo se realizará en Notion mediante un tablero dividido por tareas, estados y sprints.

La aplicación se desarrollará en Java utilizando Android Studio.

Las herramientas previstas son las siguientes:

Herramienta	Uso dentro del proyecto
GitHub	Control de versiones, repositorio de código y documentación versionada
Notion	Gestión ágil de tareas, sprints, responsables y seguimiento
Android Studio	Desarrollo de la aplicación Android
Java	Lenguaje principal de programación
Figma	Creación del prototipo inicial de pantallas

3. Distribución de permisos de los usuarios

El proyecto será desarrollado de forma individual por Luis Moyano, que será el administrador del repositorio.

Permisos reales

Usuario	Permiso	Descripción
Luis Moyano	Administrador	Puede crear ramas, subir código, modificar documentación, realizar commits y gestionar el repositorio
Profesor	Consulta	Podrá consultar el repositorio y la documentación del proyecto cuando sea necesario

Aunque el desarrollo real lo realiza una única persona, se simulan diferentes perfiles de trabajo para reflejar una organización ágil del proyecto.

Perfiles simulados

Perfil	Responsabilidad
Jefe de proyecto / Product Owner	Definir la visión del producto, priorizar funcionalidades y decidir el alcance del MVP
Scrum Master	Organizar el seguimiento del sprint, mantener actualizado el tablero de Notion y detectar bloqueos
Analista	Revisar historias de usuario, definir criterios de aceptación, flujo de navegación y modelo de datos
Desarrollador Android	Implementar la aplicación en Android Studio
Responsable de repositorio	Gestionar GitHub, ramas, commits y estructura inicial del repositorio
Tester / QA	Definir y realizar pruebas básicas

4. Política de actualización de código

Para mantener el repositorio organizado se seguirá una política sencilla de ramas y commits.

El objetivo es separar el código estable, el desarrollo activo y la documentación del proyecto. De esta forma, aunque el proyecto sea individual, se mantiene una estructura similar a la de un entorno profesional.

4.1 Política de ramas

Se utilizarán las siguientes ramas:

Rama	Uso
main	Rama principal. Contendrá versiones estables del proyecto
develop	Rama de desarrollo. Se utilizará para implementar y probar la aplicación Android antes de pasar cambios estables a main
documentacion	Rama destinada a almacenar la documentación del proyecto
feature/nombre-funcionalidad	Ramas opcionales para desarrollar funcionalidades concretas

Rama main

Será la rama principal del repositorio. Contendrá únicamente versiones estables del proyecto.

No se trabajará directamente sobre esta rama salvo para integrar cambios ya revisados.

Rama develop

Será la rama de desarrollo principal. En ella se integrarán los cambios relacionados con la aplicación Android antes de pasarlos a una versión estable.

Rama documentacion

Será la rama destinada a la documentación del proyecto. En esta rama se subirán los documentos generados durante el desarrollo, como la propuesta, el prototipo, el modelo de datos, la estrategia de pruebas o el documento de gestión del repositorio.

En esta fase inicial, la documentación se ha trabajado en una rama separada para no mezclar documentos de gestión con el desarrollo de código de la aplicación.

Ramas feature

Se podrán crear ramas de funcionalidad si una tarea concreta lo requiere.

Ejemplos:

feature/login
feature/rutinas
feature/entrenamientos

4.2 Flujo de trabajo previsto

El flujo de trabajo será el siguiente:

1. Las tareas se planifican en Notion.
2. Se selecciona una tarea del sprint correspondiente.
3. Si la tarea es de documentación, se trabaja en la rama documentacion.
4. Si la tarea es de desarrollo, se trabaja en develop o en una rama feature.
5. Se realizan commits pequeños y descriptivos.
6. Se comprueba que el proyecto compila correctamente antes de subir cambios de código.
7. Cuando una parte esté revisada y estable, podrá integrarse en main.

5. Política de commits

Los commits deberán ser claros, concretos y descriptivos.

Cada commit representará un cambio concreto del proyecto. Se evitarán commits demasiado grandes o que mezclen varias funcionalidades distintas.

Normas principales

- Los commits deben describir claramente el cambio realizado.
- Se evitará subir código que impida compilar el proyecto.
- Siempre que sea posible, los commits estarán relacionados con tareas del tablero de Notion.
- La rama main solo recibirá cambios estables.
- La documentación también se versionará mediante commits.
- No se mezclarán cambios de documentación y cambios de código en un mismo commit salvo que estén directamente relacionados.

Ejemplos de commits

Inicializa proyecto Android

Añade README inicial

Añade prototipo inicial

Crea modelo de datos

Implementa creación de rutina

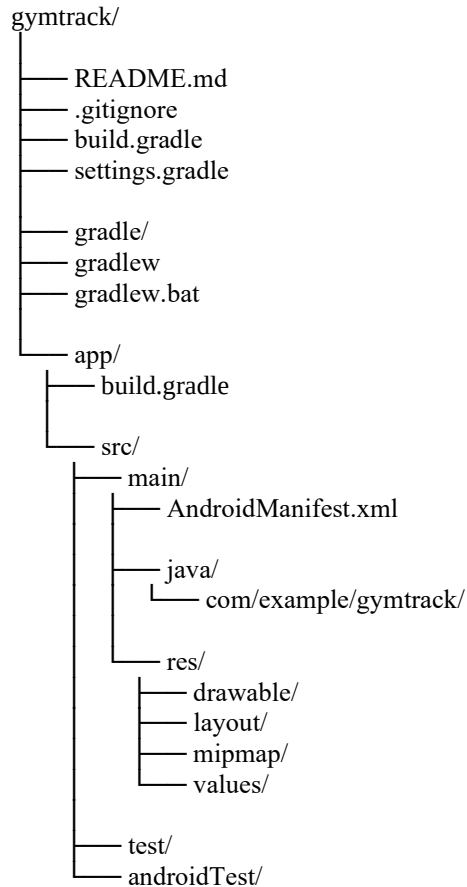
Corrige validación de rutina sin nombre

6. Estructura inicial del código fuente

La estructura inicial del código fuente corresponde al proyecto Android generado con Android Studio.

En esta fase inicial, el repositorio contiene la base del proyecto Android. La documentación principal se gestiona en una rama separada llamada documentacion.

Estructura inicial del proyecto Android



Nota: dependiendo de la versión de Android Studio, algunos archivos pueden aparecer como build.gradle.kts y settings.gradle.kts en lugar de build.gradle y settings.gradle.

7. Estructura de documentación

La documentación del proyecto se ha generado y organizado en una rama específica llamada `documentacion`.

Esta decisión permite separar la documentación del desarrollo principal de la aplicación Android.

La estructura prevista de documentación es:

```
gymtrack/  
├── docs/  
│   ├── propuesta-proyecto.pdf  
│   ├── prototipo.pdf  
│   ├── modelo-datos.pdf  
│   ├── estrategia-pruebas.pdf  
│   ├── gestion-repositorio.pdf  
│   └── retrospectiva-sprint-1.pdf
```

En esta fase no se han creado actas de reunión dentro del repositorio. Las reuniones y el seguimiento del proyecto se gestionan principalmente desde Notion y mediante la revisión con el profesor.

Si durante el Sprint 2 fuese necesario documentar reuniones de forma específica, se podría añadir posteriormente una carpeta `docs/reuniones/`.

8. Descripción de carpetas y archivos principales

Elemento	Descripción
README.md	Archivo principal del repositorio. Incluye descripción general del proyecto y tecnologías utilizadas
.gitignore	Archivo para excluir del repositorio ficheros generados automáticamente o innecesarios
build.gradle / build.gradle.kts	Archivo de configuración general del proyecto Android
settings.gradle / settings.gradle.kts	Archivo de configuración de módulos del proyecto
gradle/	Carpeta con archivos necesarios para Gradle
gradlew / gradlew.bat	Scripts para ejecutar Gradle
app/	Módulo principal de la aplicación Android
AndroidManifest.xml	Archivo de configuración principal de la aplicación Android
java/com/example/gymtrack/	Carpeta donde se ubicará el código Java de la aplicación
res/	Carpeta de recursos visuales de Android
layout/	Carpeta donde se ubicarán los diseños XML de las pantallas
drawable/	Carpeta de imágenes y recursos gráficos
values/	Carpeta de colores, textos, temas y estilos
test/	Carpeta para pruebas unitarias
androidTest/	Carpeta para pruebas instrumentadas de Android
docs/	Carpeta de documentación del proyecto, ubicada en la rama documentacion

9. Relación con la metodología ágil

El proyecto se organiza siguiendo una metodología ágil simulada, dividida en sprints.

Sprint 0

Se centró en la propuesta inicial del proyecto, definición del MVP, primeras historias de usuario y creación del tablero de gestión en Notion.

Sprint 1

Se ha centrado en la preparación del proyecto antes de la implementación. Durante este sprint se han trabajado tareas de análisis, documentación, prototipo, flujo de navegación, modelo de datos, estrategia de pruebas, repositorio y estructura inicial.

Sprint 2

Se centrará en la implementación de la aplicación Android, creación de clases, pantallas principales, funcionalidades del MVP, pruebas básicas y documentación final.

Notion se utilizará para gestionar el seguimiento del proyecto, mientras que GitHub almacenará el código fuente y la documentación versionada.